BLOCK SHIELD: SECURE PHISHING DETECTION USING MACHINE LEARNING AND BLOCKCHAIN

A PROJECT REPORT

Submitted by

MADHAN KUMAR B. **(2116231901028)**

KEERTHANA S. **(2116231901022)**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "BLOCK SHIELD: SECURE PHISHING DETECTION USING MACHINE LEARNING AND BLOCKCHAIN" is the bonafide work of "**MADHAN KUMAR B. (2116231901028), KEERTHANA S. (2116231901022)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

6/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Rohini. C
06/10/26

SIGNATURE

Ms. C. Rohini

SUPERVISOR

ASSISTANT PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

6/10/2026

External Examiner

ABSTRACT

Phishing websites are fake websites that steal users' personal and financial information by appearing as legitimate websites. Traditional phishing detection methods cannot effectively identify newly created phishing websites. This project proposes a Machine Learning-Based Phishing Website Detection Using Blockchain and Threat Intelligence system to improve phishing detection and security. The system analyzes URL and website features using a machine learning model to classify websites as phishing or legitimate. It also integrates threat intelligence from trusted security sources to enhance the detection of known malicious websites. The detection result is securely stored on the blockchain, where it cannot be changed or deleted, ensuring transparency, data integrity, and trust. By combining machine learning, threat intelligence, and blockchain technology, the proposed system provides accurate phishing detection and secure result verification, helping protect users from evolving online phishing attacks.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. C. Rohini, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

MADHAN KUMAR B 211623190128

KEERTHANA S 2116231901022

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	9
	1.2 PROBLEM STATEMENT	10
	1.3 OBJECTIVES	10
	1.4 SCOPE	11
	1.5 SECURITY REQUIREMENTS	11
	1.6 SDG ALIGNMENT	
2	**LITERATURE REVIEW**	
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	13
	2.1.1 BLACK-LIST AND REPUTATION LISTS	13
	2.1.2 RULE – BASED AND HEURISTIC DETECTION	13
	2.1.3 MACHINE LEARNING ON URL FEATURES	13
	2.1.4 CONTENT-BASED AND THIRD-PARTY ANALYSIS	14
	2.1.5 LOGGING OF DETECTION RESULTS	14
	2.2 RESEARCH GAPS	15
	2.3 PROPOSED SYSTEM	15
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	15
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	16

	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	16
3	**SYSTEM DESIGN**	
	3.1 SYSTEM ARCHITECTURE	17
	3.1.1 SOFTWARE AND HARDWARE REQUIREMENTS	18
	3.2 MODULE DESIGN	19
	3.2.1 ENFORCEMENT AND USER INTERFACE	19
	3.2.2 DECISION ENGINE	20
	3.2.3 ALLOWLIST	20
	3.2.4 LEDGER	20
	3.2.5 PERSISTENCE	21
	3.3 UML DIAGRAMS	21
	3.4 THREAT MODEL AND ATTACK SURFACE	23
	3.5 SECURITY CONTROLS	24
4	**SYSTEM DESCRIPTION**	
	4.1 DATA COLLECTION	25
	4.2 DATA PREPROCESSING	25
	4.2.1 ALGORITHMS	27
	4.2.2 RESULTS AND DISCUSSION	28
5	**CONCLUSION**	32
	REFERENCES	33

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.6	SDG ALIGNMENT OF THE PROJECT	10
2.1	COMPARISON OF THE COMMON PHISHING DEFENCE APPROACHES	14
2.2	LITERATURE SURVEY OF PUBLISHED WORK	14
3.1	CHROME EXTENSION CONFIGURATION	17
3.2	REST ENDPOINTS OF THE FLASK BACKEND	18
3.3	SOFTWARE REQUIREMENTS	18
3.4	HARDWARE REQUIREMENTS	18
3.5	MODULES OF BLOCK SHIELD	19
3.6	DECISION RULES APPLIED BY /SCAN (IN ORDER)	20
3.7	SECURITY CONTROLS: IMPLEMENTED AND NOT IMPLEMENTED	24
4.1	DATA FILES IN THE BACKEND	25
4.2	URL FEATURES USED BY THE CLASSIFIER	26
4.3	TRAINING CONFIGURATION	27
4.4	EVALUATION ON THE HELD-OUT TEST SET(40,378 URLs)	29
4.5	TEST CASES PLANNED FOR THE FINAL REPORT	31

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	17
3.2	HASH-CHAIN LEDGER STRUCTURE	21
3.3	USE CASE DIAGRAM	21
3.4	ACTIVITY DIAGRAM	22
3.5	SEQUENCE DIAGRAM	23
4.1	DATASET FLOW DIAGRAM	26
4.2	CONFUSION MATRIX DIAGRAM	29
4.3	FEATURE IMPORTANCE OF THE RANDOM FOREST	30
4.4	EXTENSION POPUP SHOWING VERDICT AND SCORE	31
4.5	WARNING PAGE SHOWN AFTER A BLOCH VERDICT	32
4.6	TERMINAL OUTPUT	32

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Phishing is still one of the easiest ways for an attacker to get into a person's accounts. The attacker does not have to break any software. A link is sent through an email, an SMS or a chat message, the link opens a page that looks like a bank, a payment service or a login screen the victim already knows, and the victim types in a password or card detail. By the time anything looks wrong the data has already been submitted.

Most browsers depend on blacklists to stop this. A blacklist works well for a page that has already been reported and checked, but a freshly registered phishing domain will not be on it yet, and the gap between the first victim and the listing can be long. Machine learning classifiers try to close that gap by scoring a URL the moment it is seen. Their weakness is different: they are only as reliable as the data used to train them.

Block Shield was built around both of these observations. It is a Chrome extension working together with a local Flask server. Each time the user navigates to a page, the extension asks the server for a verdict. The server first checks a hash-chained ledger of URLs it has already blocked, then a list of well-known trusted domains, and only after that passes the URL to a Random Forest classifier. If the verdict is Block, the extension replaces the page with a warning screen before the phishing page is displayed.

The project also has a history that shaped its design. An earlier version was trained on artificially generated phishing URLs and on legitimate URLs that were all bare home pages. That model picked up the wrong rule, that a URL with a path is suspicious, and it started flagging normal links such as a repository page on GitHub. The current version was rebuilt to avoid that mistake, and this rebuilt version is what is described in this report.

1.2 PROBLEM STATEMENT

Blacklist-based protection reacts only after a phishing site has been reported, so new sites remain unprotected for some time. Classifiers that look at the URL alone can

react immediately, but when the training data is unrealistic they produce two kinds of damage: they block genuine websites, and they miss phishing URLs that do not resemble the training samples. A user who is repeatedly blocked from genuine sites will soon turn the tool off. In addition, a detection tool that keeps a local record of the URLs it has blocked gives no way of telling whether that record was changed afterwards.

The problem this project addresses is therefore to decide, before a page is displayed, whether a URL is likely to be phishing, using a model trained on real data, while keeping false alarms on popular sites low and keeping a log of blocked URLs that shows any later modification.

1.3 OBJECTIVES

The main objective is to detect phishing URLs in real time inside Chrome and block them automatically, using a locally trained classifier, a trusted-domain list and a tamper-evident ledger. The specific objectives are listed below.

- To compute 16 lexical and structural features from a URL without sending any request to the destination site.
- To classify these features with a Random Forest and produce a phishing probability between 0 and 1.
- To mark URLs from well-known domains as Safe directly, so that ordinary deep links are not flagged by the model.
- To block URLs that were already confirmed as phishing earlier, by looking them up in the ledger.
- To store every confirmed Block in a SHA-256 hash-chained ledger whose consistency can be verified in one pass.
- To redirect the browser tab to a warning page before a blocked page is rendered.
- To keep a database record of every scan and show the current verdict, score and recent history in a popup.

1.4 SCOPE

The system works on the URL string of top-level navigations in Chrome and other Chromium-based browsers that support Manifest V3. It runs completely on the user's machine: the extension, the Flask server, the model, the ledger file and the database are all local. The database can be SQLite, which needs no setup, or MySQL 8.0.

Several things are outside the scope of the present work. The page content, its HTML and its visual appearance are not analysed. No external threat-intelligence service is called, so VirusTotal, URLScan or a live phishing feed is not part of the implementation. The ledger is a single-node hash chain and not a distributed blockchain, and it has no smart contract. The Flask API has no authentication, and the system is designed for one user on one machine. These points are discussed again in Chapters 3 and 5.

1.5 SECURITY REQUIREMENTS

The requirements below were considered while designing the system. Whether each of them was actually met is discussed in Section 3.5.

- Integrity: a record of a blocked URL should not be alterable without the change being detectable.
- Availability: if the backend is not running, browsing must not freeze or be blocked for every site.
- Privacy: the URL should be analysed on the user's machine, without sending it to a third-party service.
- Low false-alarm rate: genuine websites should not be blocked, otherwise the user will stop trusting the tool.
- Timeliness: the verdict must arrive before the destination page is displayed.
- Safe handling of input: URLs come from the outside world, so database queries and page rendering must not trust them.

1.6 SDG ALIGNMENT

The mapping below is our own reading of the goals and should be confirmed with the project guide before submission.

Table 1.1 SDG alignment of the project

S.No.	SDG No. and goal	Relevant SDG target	Project contribution / alignment
1	SDG 9: Industry, Innovation and Infrastructure	**Target 9.5:** Enhance scientific research and technological capabilities, particularly in developing countries.	Adds a locally run protection layer to everyday web browsing, and builds on open-source tools such as Flask and scikit-learn.
2	SDG 16: Peace, Justice and Strong Institutions	**Target 16.4:** Significantly reduce illicit financial and arms flows, strengthen the recovery and return of stolen assets, and combat all forms of organized crime.	Reduces the chance of fraud through phishing links by blocking suspected URLs, and keeps a checkable record of what was blocked.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Phishing defence has been approached in several ways. This section describes the main approaches in general terms and points out where each one is weak. The published papers we studied for the formal literature survey are to be entered in Table 2.2.

2.1.1 Blacklist and reputation lists

Browser vendors and community projects maintain lists of pages that were reported and verified as malicious. Google Safe Browsing and PhishTank are well-known examples. When a URL is found on such a list the browser can warn the user immediately. The strength of this approach is precision, because every entry has been checked. The weakness is timing: a new phishing site is unknown until it is reported and verified.

2.1.2 Rule-based and heuristic detection

Heuristic systems apply hand-written rules, for example flagging a URL that contains an IP address in place of a domain name, or an "@" sign in the middle. The rules are easy to explain and cheap to run. However, an attacker who knows the rules can simply avoid them, and each new trick needs a new rule.

2.1.3 Machine learning on URL features

A classifier can learn from thousands of labelled URLs which combinations of length, symbols, subdomains and keywords tend to belong to phishing. Because only the URL string is needed, the verdict can be given before the page loads. The difficulty, which we met directly in our first version, is the training data. When the two classes differ in some accidental way, for instance all legitimate URLs being home pages, the model learns that accident instead of real phishing behaviour.

2.1.4 Content-based and third-party analysis

Some systems load the page and compare its content or appearance with the brand it seems to imitate, and services such as VirusTotal or URLScan combine many engines

or render the page remotely. These can catch phishing that has an ordinary-looking URL. They also cost time, need network access to a third party, and send the visited URL outside the user's machine. Block Shield does not use these services.

2.1.5 Logging of detection results

Local security tools generally keep their detection log as a database table or a text file, which anyone with file access can edit without leaving a trace. Hash-chaining, the technique behind blockchains, is a light way to make such edits detectable. Block Shield applies this idea to its record of blocked URLs, but only in a single-node form.

Table 2.1 Comparison of common phishing defence approaches

Approach	Main strength	Main weakness
Blacklist / reputation list	High precision on listed pages	New sites are unknown until reported
Hand-written heuristics	Simple and explainable	Attackers can avoid fixed rules
ML on URL features	Scores unseen URLs instantly	Depends heavily on training data quality
Content or third-party analysis	Can catch well-disguised pages	Slower, needs external services, shares the URL
Plain local log	Easy to implement	Edits cannot be detected

Table 2.2 Literature survey of published work

S.No.	Author and year	Method used	Findings	Limitation
1	J. Ma et al. (2009)	Supervised machine learning using lexical and host-based URL features for malicious website detection.	Demonstrated that suspicious URL characteristics can be used to identify malicious websites without relying only on blacklists.	Detection performance depends on the quality and representativeness of the training data.
2	O. K. Sahingoz et al. (2019)	Machine-learning-based phishing detection using URL-derived features and multiple classification algorithms	Compared several classifiers and reported Random Forest as the strongest classifier on their selected feature set.	Performance depends on the selected dataset and URL features; generalization to changing phishing patterns remains a concern.

S.No.	Author and year	Method used	Findings	Limitation
3	S. Gong and C. Lee (2020)	Blockchain-based cyber threat intelligence sharing framework (BLOCIS).	Proposed blockchain for sharing cyber-threat intelligence while providing a tamper-resistant and distributed record of threat information.	Requires cooperation among multiple participants and introduces the complexity of a distributed blockchain infrastructure.

The dataset used in this project comes from the PhiUSIIL Phishing URL Dataset (Prasad and Chandra, 2024), which is the only published study we have cited so far. Further papers should be added to Table 2.2 and to the reference list by the team.

2.2 RESEARCH GAPS

From the approaches above and from our own first attempt, we identified the following gaps that this project tries to handle.

- Classifiers trained on synthetic or unrealistic URLs give false alarms on ordinary deep links.
- Blacklists alone leave a gap for newly created phishing sites.
- Third-party analysis services send the visited URL to an outside party and add delay.
- Locally stored detection records usually cannot show whether they were modified.

2.3 PROPOSED SYSTEM

2.3.1 Overview of the Proposed System

Block Shield has two parts. The Chrome extension watches top-level navigations and enforces the result. The Flask backend takes the URL and decides. The backend applies three checks in a fixed order and stops at the first one that gives an answer: the hash-chained ledger, the trusted-domain allowlist, and finally the Random Forest model. The model output is compared with two thresholds, 0.60 for Block and 0.30 for Warning, and everything below 0.30 is Safe.

Every scan is written to a database table, and every Block verdict is also appended to the ledger. The popup shows the verdict, the model score, the backend status and the last few scans.

2.3.2 Objectives of the Proposed System

- Give a verdict for a URL before the page is displayed.
- Reduce false alarms on popular sites by using a domain allowlist together with the model.
- Train the model on real phishing and legitimate URLs instead of synthetic ones.
- Make later changes to the record of blocked URLs detectable.
- Work without any API key or external service.

2.3.3 Advantages of the Proposed System

- The analysis runs locally, so the visited URL is not sent to a third party.
- No request is made to the destination site during feature extraction.
- The allowlist is meant to keep well-known domains away from the model, which was the source of most false alarms in the first version.
- A URL that was blocked once is blocked instantly on the next visit through the ledger.
- The ledger can be checked at any time through the /ledger/verify endpoint.
- If the backend is down, the extension shows an Unknown status and does not stop browsing.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture is a simple client–server arrangement in which both sides run on the same computer. Figure 3.1 shows the components. The extension has a service worker (background.js), a popup, and a warning page. The service worker is the only part that can enforce a decision, because it is the one that redirects the tab. The backend contains the API and decision logic in app.py, the feature and allowlist code in features.py, the ledger in blockchain.py, the database layer in db.py and the settings in config.py.

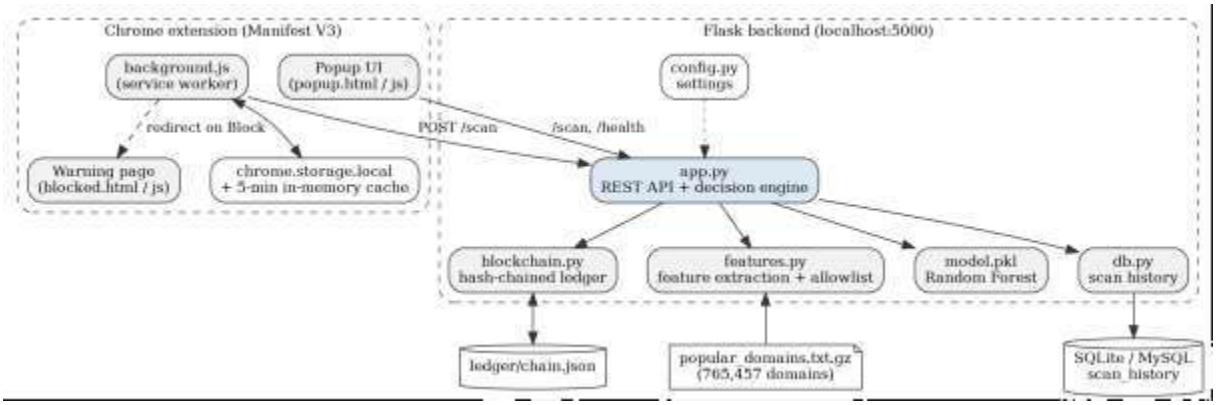


Figure 3.1 System architecture of Block Shield

The extension talks to the backend over plain HTTP at localhost port 5000. It listens to the chrome.webNavigation.onBeforeNavigate event and handles only top-level frames (frameId 0), so iframes on a page are not scanned. Results for a URL are kept in a five-minute in-memory cache inside the service worker, and the last result is also saved in chrome.storage.local so that the popup can read it.

Table 3.1 Chrome extension configuration

Item	Value in manifest.json
Manifest version	3
Permissions	webNavigation, tabs, storage, activeTab
Host permissions	http://*/*, https://*/*, http://localhost:5000/*
Background	background.js as service worker
Popup	popup.html

Item	Value in manifest.json
Content scripts	None
Web-accessible resources	blocked.html, blocked.css

Table 3.2 REST endpoints of the Flask backend

Endpoint	Method	Purpose
/scan	POST	Takes {"url": ...} and returns the verdict, score and source
/history	GET	Returns recent scans; optional limit parameter, default 50
/health	GET	Reports model status, ledger length and validity, database type
/ledger/verify	GET	Runs the hash-chain check and returns valid and length

3.1.1 Software and Hardware Requirements

Table 3.3 Software requirements

Component	Version / detail
Python	3.x (minimum version not fixed in the project)
Flask / Flask-CORS	3.0.3 / 4.0.1
scikit-learn / pandas / numpy / joblib	1.5.1 / 2.2.2 / 1.26.4 / 1.4.2
tldextract	5.1.2
mysql-connector-python (optional)	9.0.0
Database	SQLite (default) or MySQL 8.0
Browser	Google Chrome or another Chromium browser with Manifest V3 support

Table 3.4 Hardware requirements

Component	Specification
Processor	AMD Ryzen 5 8645HS with Radeon 760M Graphics
RAM	16 GB
Storage	512 GB SSD
Operating system	Windows 11 64-bit

The project files do not state a hardware requirement. The trained model file is about 23.6 MB and the training data is a few hundred thousand short strings, so no special hardware should be needed, but the machine actually used for development and testing has to be entered in Table 3.4.

3.2 MODULE DESIGN

The system is divided into the modules summarised in Table 3.5. The decision module is the centre; the others either feed it or act on what it returns.

Table 3.5 Modules of Block Shield

Module	Files	Role
Enforcement	background.js, manifest.json	Catches navigation, calls /scan, sets the badge, redirects on Block
Popup	popup.html / css / js	Shows verdict, score meter, backend and ledger status, recent scans
Warning page	blocked.html / css / js	Full-page notice with go-back and proceed options
Decision engine	app.py	Runs ledger, allowlist and ML checks and applies thresholds
Feature extraction	features.py	Builds the 16-value feature set and holds the allowlist logic
ML classifier	train_model.py, model.pkl	Trains offline; scores URLs at run time
Ledger	blockchain.py	Hashing, appending, lookup and validation of the chain
Persistence	db.py	Creates the scan_history table, inserts and reads rows
Configuration	config.py, .env.example	Thresholds, database type, host, port and debug flag from environment variables

3.2.1 Enforcement and user interface

When a navigation starts, handleNavigation() in background.js ignores everything except real web pages in the top frame and asks scanUrl() for a verdict. Depending on the result it sets a badge colour on the toolbar icon. For Warning it also shows a desktop notification and lets the page load. For Block it redirects the tab to blocked.html, passing the URL, the score and the source as query parameters, and shows a notification. The warning page then displays these values and offers two buttons, one to go back and one to proceed anyway. If the request to the backend fails, scanUrl() returns an Unknown verdict, so the user can continue to browse.

3.2.2 Decision engine

The scan() function in app.py is the core of the backend. Browser-internal addresses such as chrome:// and about: are returned as Safe without any analysis. The other URLs go through the checks in Table 3.6.

Table 3.6 Decision rules applied by /scan (in order)

Order	Condition	Verdict	Source label
0	Browser-internal address (chrome://, about:, edge://, brave://)	Safe	skipped
1	SHA-256 of the URL is present in the ledger	Block	ledger
2	Registrable domain is in the allowlist and not in the exclusion list	Safe	allowlist
3	ML score >= 0.60	Block	full_analysis
4	0.30 <= ML score < 0.60	Warning	full_analysis
5	ML score < 0.30	Safe	full_analysis

The label full_analysis therefore means model-only analysis. Ledger and allowlist results are returned before the model runs. Both thresholds can be changed through environment variables without editing the code.

3.2.3 Allowlist

The allowlist is a gzip text file of 765,457 popular registrable domains, sampled from the Chrome UX Report and bundled with the project. It is loaded into a set, so a lookup takes constant time on average. A separate exclusion list removes URL shorteners and free hosting platforms from the fast path, because a popular domain of that kind can still be used to hide a phishing page. A code comment in app.py also admits that the allowlist cannot catch an open redirect or a script injection hosted on a popular domain itself.

3.2.4 Ledger

blockchain.py keeps a list of blocks in the file ledger/chain.json. Each block (Figure 3.2) has an index, the SHA-256 hash of the URL, the hash of the previous block, a timestamp and its own hash. The first block, the genesis block, is created automatically when the file is first used. The repository does not include a chain.json, so the ledger

starts empty on a new installation. A threading lock protects concurrent writes, and the file is saved by writing a temporary file and then renaming it, so an interrupted write does not leave a half-written chain.

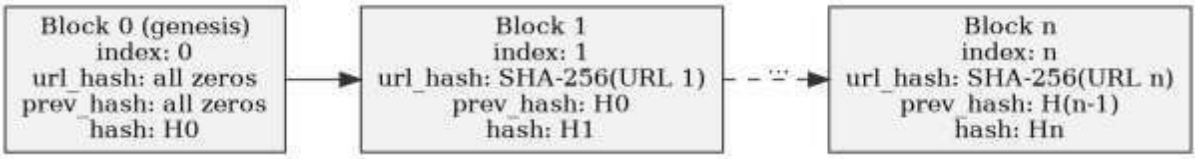


Figure 3.2 Structure of the hash-chained ledger

3.2.5 Persistence

db.py exposes the same functions for both database types. It has a single table, scan_history, with the columns id, url, ml_score, verdict and scanned_at. The only operations implemented are inserting a row and reading the latest rows; nothing is updated or deleted. All queries use parameter placeholders.

3.3 UML DIAGRAMS

The use case diagram in Figure 3.3 shows the two kinds of users. The ordinary user only browses and reads the results. The administrator is the person who prepares the data, trains the model, sets the configuration and can verify the ledger.

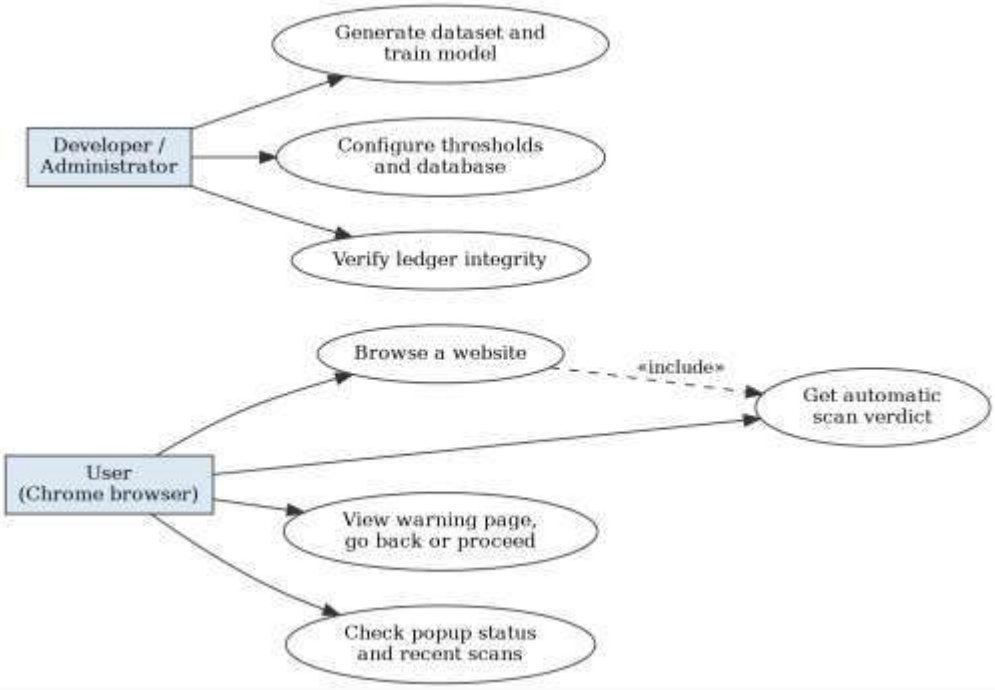


Figure 3.3 Use case diagram

Figure 3.4 is the activity diagram of one scan. It follows the same order as Table 3.6. The scan is written to the database in every case that reaches the end of the analysis, and the ledger is written only when the result is Block.

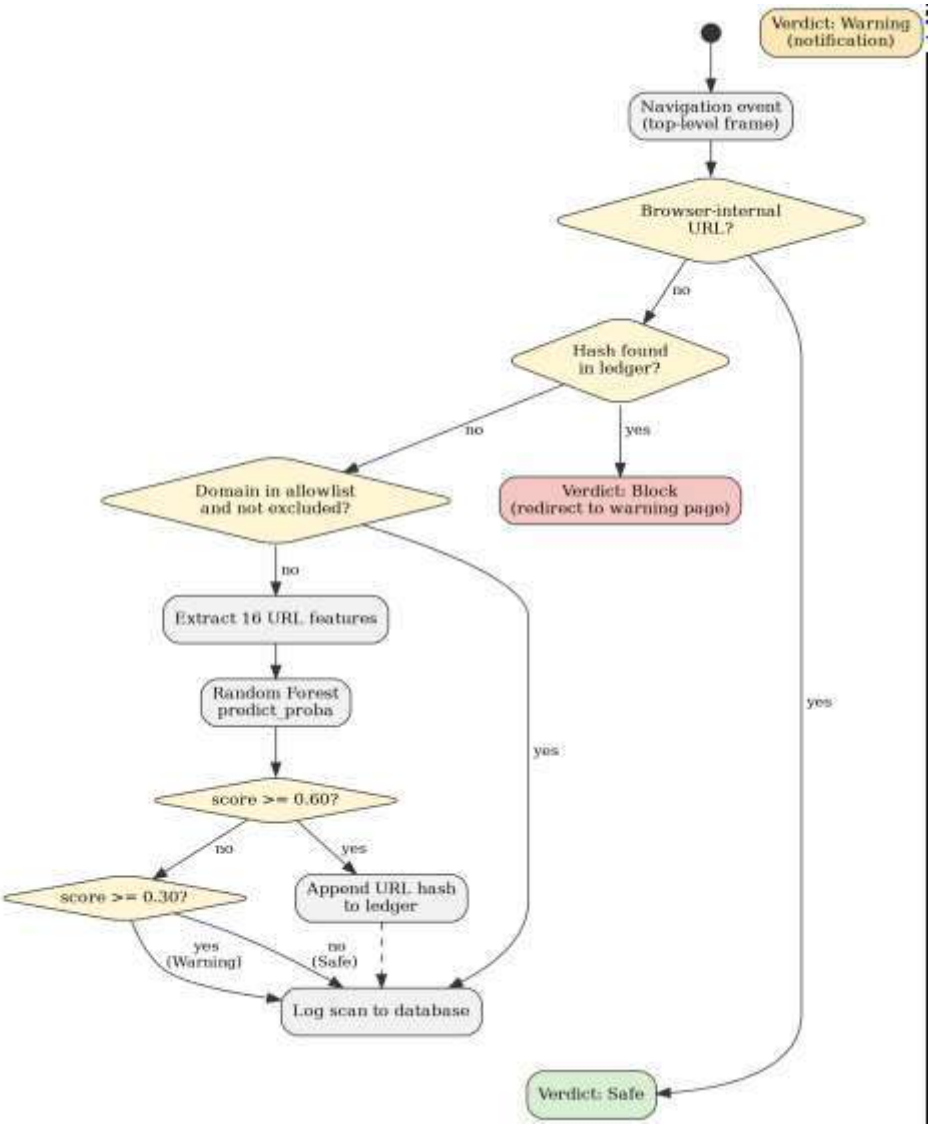


Figure 3.4 Activity diagram of a single scan

The sequence of messages between the browser, the extension and the backend components is shown in Figure 3.5. In the figure the allowlist and the model are drawn as one lifeline for readability, although in the code the allowlist is checked first and the model runs only when it does not match.

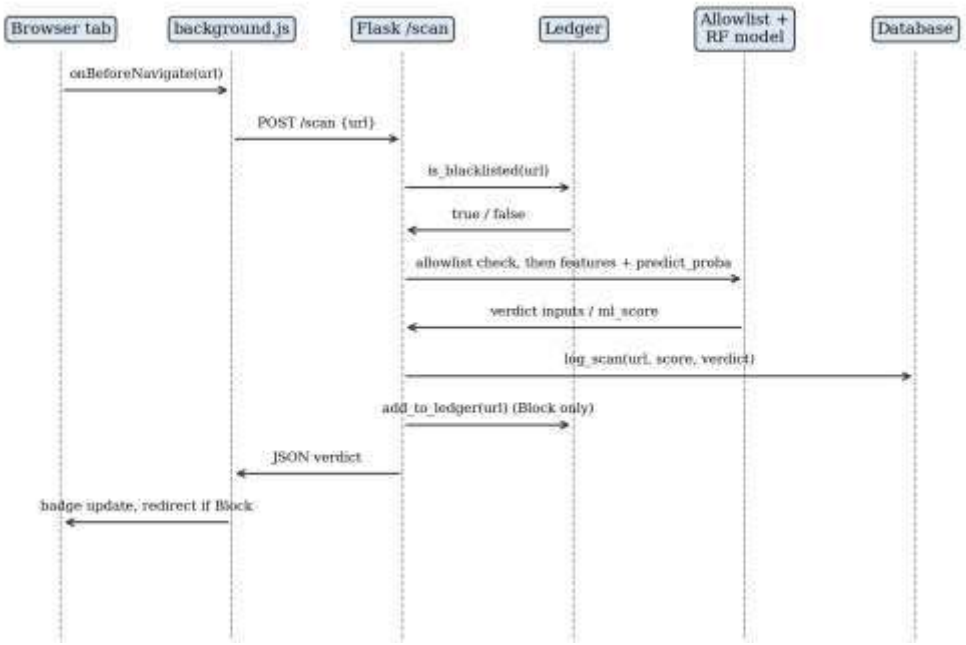


Figure 3.5 Sequence diagram for a navigation event

3.4 THREAT MODEL AND ATTACK SURFACE

The assets to protect are the integrity of the ledger and the scan history, the trained model file, and the URLs the user visits as they pass through the local pipeline. We considered three kinds of actors: an attacker running a phishing site who tries to avoid detection, a local program with access to the user's files, and any other process or web page that can reach the Flask port.

While reviewing our own code we found the following weak points. We list them here because they are part of the real state of the project.

- The API endpoints have no authentication. The CORS setting is CORS(app) with no origin argument, so any web origin is allowed, even though a comment in the code suggests it is limited to the extension.
- config.py defaults HOST to 0.0.0.0 and DEBUG to true. On a shared network the service could therefore be reachable from other machines, and the debug mode is not appropriate outside development.
- ledger/chain.json and the database file are plain files. Editing them is possible; the hash chain only makes an edit to the ledger detectable, and only when /ledger/verify is run.

- In popup.js the function addHistoryItem() builds part of the history item through innerHTML using the URL text without escaping it, which is a possible DOM injection point in the popup.
- The ledger key is a hash of the exact URL string. A change of scheme, a trailing slash or a query parameter produces a different hash, so those variants do not match the ledger. They are still scored by the model.
- The decision uses the URL only. A phishing page with a normal-looking URL, or one hosted on a trusted domain, can pass.

3.5 SECURITY CONTROLS

Table 3.7 Security controls: implemented and not implemented

Control	Status	Where
SHA-256 hashing of URLs and blocks	Implemented	blockchain.py
Hash-chain validation (/ledger/verify)	Implemented	blockchain.py, app.py
Atomic writes to the ledger file	Implemented	blockchain.py
Parameterised SQL queries	Implemented	db.py
Settings and credentials from environment variables	Implemented	config.py, .env.example
Fail-open behaviour when backend is down	Implemented	background.js
Input validation on /scan	Partial: only checks the URL is not empty	app.py
Safe escaping in popup history	Partial: most text uses textContent, history item does not	popup.js
API authentication	Not implemented	-
Restricted CORS	Not implemented	app.py
Localhost-only default and debug off	Not implemented	config.py
HTTPS between extension and backend	Not implemented	-
Rate limiting and request logging	Not implemented	-

The controls that are not implemented are treated as future work in Chapter 5.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The URLs used for training come from the PhiUSIIL Phishing URL Dataset (Prasad and Chandra, 2024). The project keeps them as two text files, one URL per line, and the README points to a GitHub copy of the dataset. We have not independently checked this source, so the citation is as stated in the project files.

Table 4.1 Data files in the backend/data folder

File	Content	Size
real_phishing_urls.txt	Phishing URLs	100,944 lines
real_legit_urls.txt	Legitimate URLs	134,849 lines
popular_domains.txt.gz	Popular domains for the allowlist (Chrome UX Report sample)	765,457 domains
dataset.csv	Final training file with columns url and label	201,890 rows plus header

An important detail is that the legitimate URLs in the raw file were all bare home pages. Training on them would repeat the mistake of the first version, in which a path in the URL looked like a sign of phishing. The domains are therefore real, but the paths of most legitimate URLs in the final dataset were added by our own script, as described in the next section.

4.2 DATA PREPROCESSING

generate_dataset.py prepares dataset.csv in three steps. It reads the two raw URL files, it balances the classes, and it rewrites the paths of a large share of the legitimate URLs (about 70 to 90 percent) with generated slugs, numeric identifiers and query strings that look like everyday links. A fixed random seed of 42 makes the result repeatable. The script is run manually and only once per dataset; the running system never regenerates data. Figure 4.1 gives the overall flow from raw files to the trained model.

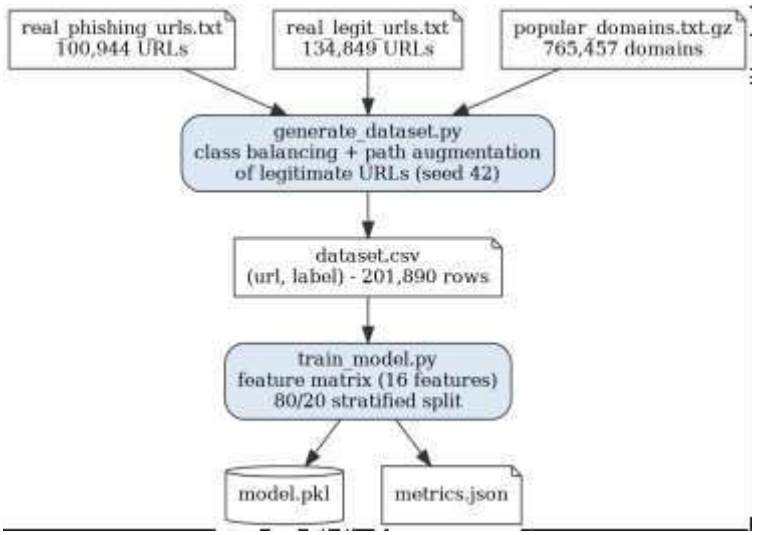


Figure 4.1 Dataset preparation and training flow

Because the legitimate paths are generated, and not collected from real browsing, the legitimate class is only partly real. We treat this as a limitation of the dataset and mention it again in the results.

Feature extraction

features.py turns a URL into 16 numeric values. The same function is used in training and at run time, so the model always sees features computed in the same way. tldextract is used to split the URL into subdomain, domain and suffix. No network request is made. Table 4.2 lists the features.

Table 4.2 URL features used by the classifier

No.	Feature	How it is computed
1	url_length	Number of characters in the URL
2	has_ip	Hostname is a dotted IPv4 address
3	has_at_symbol	URL contains "@"
4	has_https	URL starts with https
5	num_dots	Count of "."
6	num_hyphens	Count of "-"
7	num_digits	Count of digits
8	num_slashes	Count of "/"
9	has_suspicious_words	Contains a word from a fixed list, such as login, verify, secure, account
10	subdomain_count	Number of subdomain labels

No.	Feature	How it is computed
11	domain_length	Length of the registrable domain name
12	has_brand_impersonation	A known brand name appears in the URL but is not the registrable domain
13	is_shortener	Domain is in a fixed set of URL shorteners
14	num_query_params	Count of "=" plus count of "&"
15	has_double_slash_redirect	"//" appears after the scheme part
16	is_popular_domain	Domain is in the allowlist and not excluded

4.2.1 ALGORITHMS

Random Forest classifier

A Random Forest is a set of decision trees, each trained on a different random sample of the data and features, whose outputs are combined. We used the RandomForestClassifier from scikit-learn. It handles features on different scales without normalisation, and it gives feature importance values, which are useful for explaining a security decision. The settings were chosen by hand and were not tuned with a search.

Table 4.3 Training configuration

Parameter	Value
n_estimators	150
max_depth	20
min_samples_leaf	5
class_weight	balanced
random_state	42
Train / test split	80 / 20, stratified by label (161,512 / 40,378)

train_model.py loads dataset.csv, builds the feature matrix, splits it, trains the model, calculates the metrics and saves model.pkl and metrics.json using joblib. At run time, app.py loads model.pkl once at start-up and calls predict_proba() for each URL that reaches the model stage; the second column of the output, the probability of the phishing class, is the ml_score.

SHA-256 hash chain

For each Block verdict, blockchain.py lowercases and trims the URL and takes its SHA-256 digest. A new block stores this digest together with the hash of the previous block and computes its own hash over its fields. Verification reads the whole file, recomputes each block's hash and checks that every prev_hash matches the block before it. If a stored block is changed, its own hash no longer matches, and if a block is removed or replaced, the next block's link breaks. Lookup for a URL is a linear scan over the stored hashes, so its cost grows with the number of blocked URLs.

This gives tamper evidence, not tamper prevention. Someone with access to the file could rewrite the whole chain consistently, and there is no other node to compare with. For this reason we describe the ledger as blockchain-inspired, and we do not claim a distributed blockchain, a consensus method or a smart contract.

Run-time workflow

The run-time workflow is the one in Figure 3.4. A navigation event reaches background.js, which calls /scan. The backend consults the ledger, then the allowlist, then extracts features and runs the model, compares the score with the two thresholds, logs the scan, and appends to the ledger if the verdict is Block. The extension receives the JSON reply, updates the badge and either lets the page load, shows a notification, or redirects the tab.

Implementation notes

The backend is started with python app.py, which uses the Flask development server. The extension is loaded from the extension folder using Chrome's Developer mode and the Load unpacked option. No Node.js or build step is used. The database is chosen with the DB_TYPE variable, and for MySQL the schema is created automatically once the database itself exists.

4.2.2 RESULTS AND DISCUSSION

The figures in this section were taken from model/metrics.json, which is produced by train_model.py and is included in the project. We have not re-run the training for

this report, so they should be read as the values recorded by the project. Re-running python data/generate_dataset.py and python train_model.py should reproduce them closely, since seeds are fixed, although a different library version can shift them slightly.

Table 4.4 Evaluation on the held-out test set (40,378 URLs)

Metric	Value
Accuracy	98.12 %
Precision	99.41 %
Recall	96.81 %
F1-score	98.09 %
False positive rate (116 / 20,189)	about 0.57 %
False negative rate (644 / 20,189)	about 3.19 %

The last two rows were calculated by us from the confusion matrix. Each class has 20,189 test URLs, and the matrix in Figure 4.2 shows 116 legitimate URLs marked as phishing and 644 phishing URLs missed.

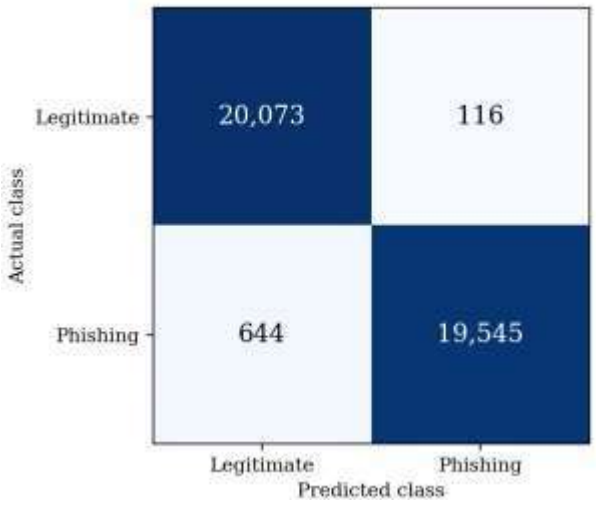


Figure 4.2 Confusion matrix on the test set

The high precision means that when the model says phishing it is almost always right, which is what matters for a tool that redirects the user away from a page. The recall is lower, so a small share of phishing URLs still pass the model. This is the same trade-off that the 0.60 Block threshold and the 0.30 Warning threshold are meant to handle, though we did not run a threshold study.

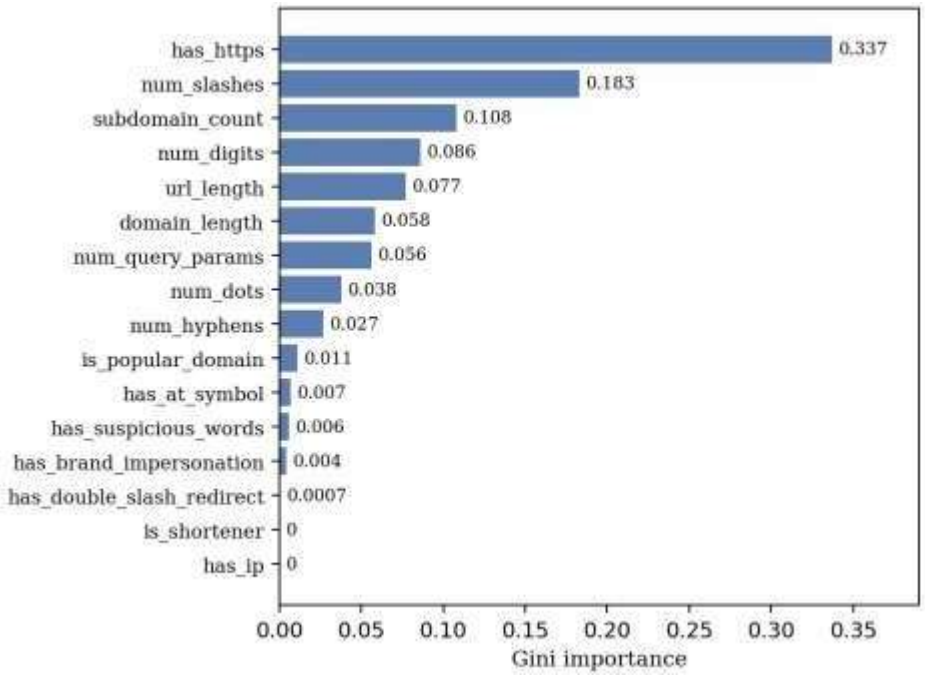


Figure 4.3 Feature importance of the Random Forest

The feature importance in Figure 4.3 shows that has_https (0.337) and num_slashes (0.183) carry the most weight, followed by subdomain_count, num_digits and url_length. has_ip and is_shortener have almost no effect on the trained model, probably because they are rare in the data. is_popular_domain has a small importance of 0.011, which is expected because popular domains are mostly settled by the allowlist before the model is used.

We should also point out that the high importance of has_https and num_slashes may reflect how the dataset was built. The legitimate paths were generated by our script, and the test set was split from the same generated file. The reported accuracy is therefore an estimate on data of the same kind, and it may not hold for real traffic. Testing on a separate set of hand-collected URLs is still to be done.

Testing status

The project has no automated test suite. What exists is manual checking with the curl commands from the README, a manual tamper test in which a byte of chain.json is edited and /ledger/verify is expected to return false, and the held-out evaluation above, which is model evaluation and not software testing. The README also

mentions an 18 out of 18 stress test, but the project has no script or log file for it, so we do not report it as a result.

Table 4.5 Test cases planned for the final report

Test	What to check	Status
Verdict paths	One URL for each of the six rows in Table 3.6	Passed
Feature extraction	Known URLs against expected feature values	Passed
Ledger	verify returns true, then false after editing chain.json	Passed
API errors	Missing url, empty body, model file missing	Passed
Database	Same behaviour on SQLite and MySQL	Passed
Extension	Badge colour, notification, redirect to warning page	Passed
Latency	latency_ms returned by /scan for a set of URLs	Passed

Screenshots

The project folder contains only the extension icons, so the screenshots below have to be taken by the team.

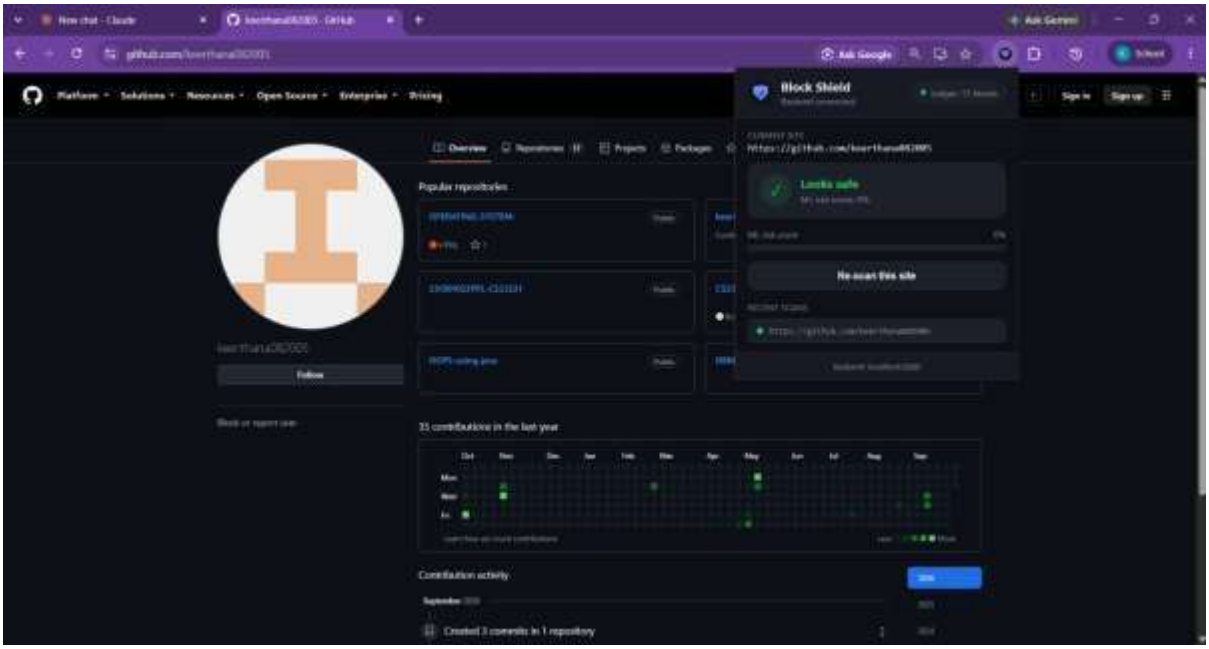


Figure 4.4 Extension popup showing verdict and score

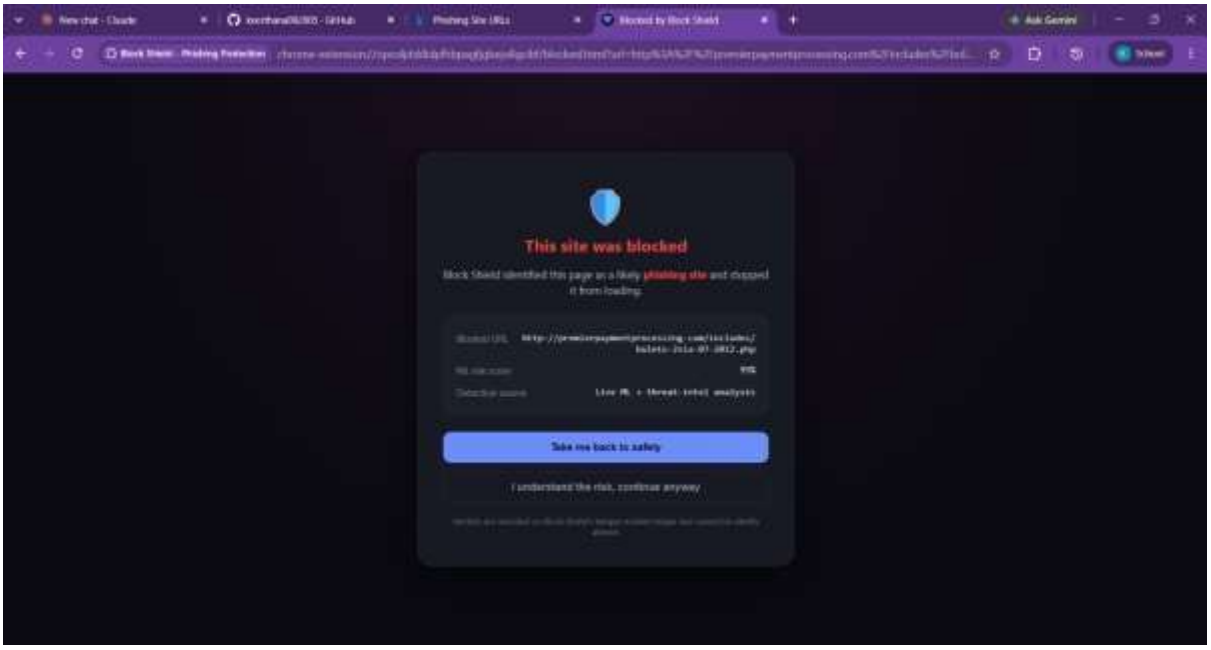


Figure 4.5 Warning page shown after a Block verdict

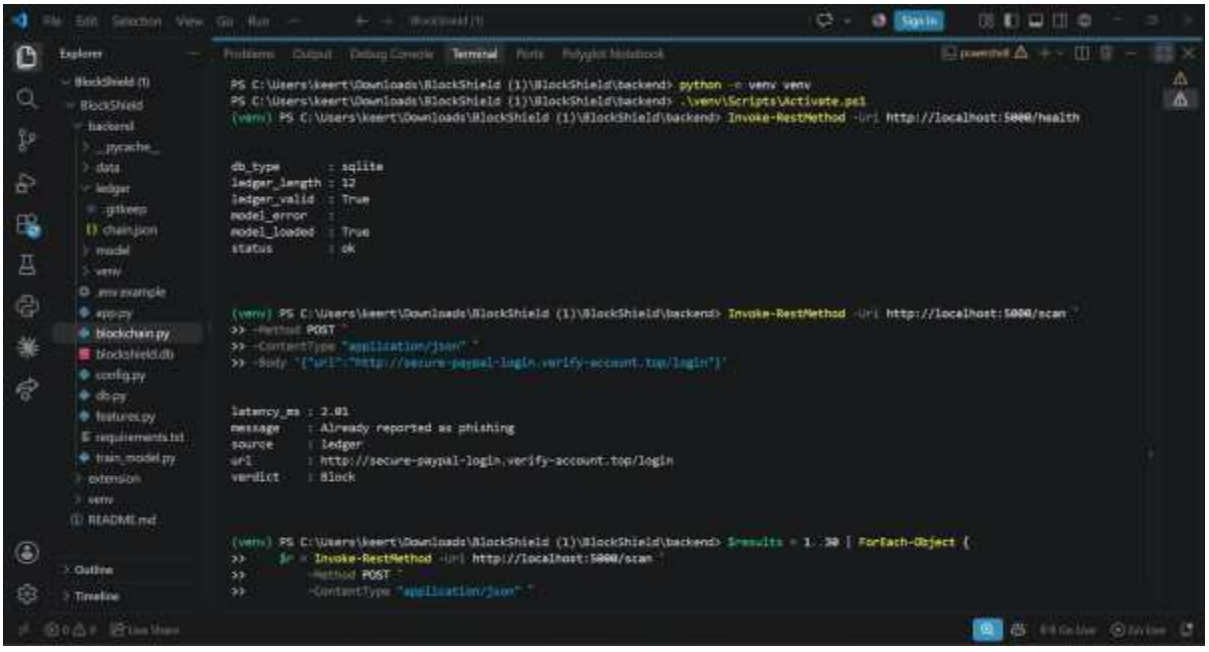


Figure 4.6 Terminal output of /health and /scan requests

CHAPTER 5
CONCLUSION

Block Shield is a working prototype that checks each page a user opens in Chrome and blocks it if the URL looks like phishing. A local Flask server takes the decision using the hash-chained ledger, a 765,457-domain allowlist and a Random Forest with 16 URL features, and the extension enforces it by redirecting the tab before the page is shown. On the held-out split recorded in metrics.json the model reached 98.12 % accuracy, 99.41 % precision and 96.81 % recall.

We think the most useful lesson from the project is about data. Our first version was trained on synthetic phishing URLs and on home-page-only legitimate URLs, and it blocked normal links. Switching to a real dataset, adding realistic paths to the legitimate class and putting the allowlist in front of the model addressed that problem in design. The ledger adds a modest but real feature: any change made to the record of blocked URLs can be detected by /ledger/verify.

The limitations are equally clear. The ledger is a single-node hash chain and not a blockchain network, and the project has no smart contract. No external threat-intelligence service is used, even though the manifest description and the warning page text mention threat-intel cross-verification; this text does not match the code and should be corrected. The API has no authentication, CORS is not restricted, the default host is 0.0.0.0 with debug on, and the popup builds one HTML fragment from an unescaped URL. There is no automated test suite, the URL hash is exact-match only, and the paths of the legitimate URLs in the dataset are generated.

REFERENCE

1. S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, and C. Zhang, “An empirical analysis of phishing blacklists,” in Proc. 6th Conf. Email and Anti-Spam (CEAS), Mountain View, CA, USA, 2009.
2. J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious URLs,” in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), Paris, France, 2009, pp. 1245–1254.
3. D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL detection using machine learning: A survey,” arXiv:1701.07179, 2017.
4. D. Arp et al., “Dos and don’ts of machine learning in computer security,” in Proc. 31st USENIX Security Symp., Boston, MA, USA, 2022, pp. 3971–3988.
5. A. Prasad and S. Chandra, “PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning,” Computers & Security, vol. 136, Art. no. 103545, 2024.
6. S. Garera, N. Provos, M. Chew, and D. A. Rubin, “A framework for detection and measurement of phishing attacks,” in Proc. ACM Workshop on Recurring Malcode (WORM), Alexandria, VA, USA, 2007, pp. 1–8.
7. A. Blum, B. Wardman, T. Solorio, and G. Warner, “Lexical feature based phishing URL detection using online learning,” in Proc. 3rd ACM Workshop on Artificial Intelligence and Security (AISec), Chicago, IL, USA, 2010, pp. 54–60.
8. C. Whittaker, B. Ryner, and M. Nazif, “Large-scale automatic classification of phishing pages,” in Proc. Network and Distributed System Security Symp. (NDSS), San Diego, CA, USA, 2010.
9. O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” Expert Systems with Applications, vol. 117, pp. 345–357, 2019.